Variabel, Tipe Data, dan Operator pada Dart

Mata Kuliah: Pemrograman Mobile

Pertemuan: 2

Capaian Pembelajaran

SUB-CPMK3

Mahasiswa mampu menerapkan pemikiran logis dan kritis dalam mengembangkan aplikasi berbasis mobile sesuai dengan berbagai macam studi kasus yang diberikan.

Detail Capaian Khusus:

- Memahami konsep variabel dan cara mendeklarasikannya dalam Dart
- Mengenali berbagai tipe data primitif dan non-primitif di Dart
- Mampu menerapkan penggunaan operator dasar di Dart
- Mampu menulis program Dart sederhana menggunakan konsep-konsep tersebut

Agenda Pembelajaran

Pengantar Dart

Memahami karakteristik bahasa pemrograman Dart

Variabel dan Deklarasi

Mempelajari konsep variabel dan cara mendeklarasikannya

Tipe Data

Mengenal tipe data primitif dan non-primitif di Dart

Operator

Mempelajari operator aritmatika, relasional, logika, dan lainnya

Pengantar Bahasa Pemrograman Dart

Apa itu Dart?

- Bahasa yang dioptimalkan untuk UI
- Dikembangkan oleh Google
- Digunakan oleh Flutter untuk pengembangan aplikasi multi-platform
- Berorientasi objek (Object-Oriented)
- Mendukung type inference
- Memiliki fitur null safety

Karakteristik Utama Dart

Berorientasi Objek

Semua nilai di Dart adalah objek, termasuk tipe primitif seperti angka dan string. Bahkan fungsi adalah objek dari tipe Function.

Kompilasi Fleksibel

JIT (Just-In-Time) untuk pengembangan cepat dengan hot reload dan AOT (Ahead-Of-Time) untuk performa aplikasi produksi yang optimal.

Null Safety

Fitur yang mencegah error akibat nilai null, membantu developer menghindari banyak bug yang sering terjadi di bahasa pemrograman lain.

Fungsi main()

Setiap program Dart dimulai dari fungsi `main()`

```
void main() {  print('Halo, Dunia!');}
```

- Titik masuk (entry point) eksekusi program
- Wajib ada di setiap program Dart yang bisa dieksekusi
- Dapat menerima parameter argumen dari command line

Variabel di Dart

Apa itu Variabel?

Sebuah tempat di memori komputer untuk menyimpan nilai yang dapat digunakan dan diubah selama eksekusi program.

Contoh:

```
var nama = 'Budi';
var umur = 20;
var tinggi = 175.5;
var aktif = true;

// Nilai variabel dapat diubah
nama = 'Ahmad';
umur = 21;
```

Cara Mendeklarasikan Variabel

Menggunakan `var`

Dart akan menebak tipe data berdasarkan nilai awal (type inference)

```
var nama = 'Budi';  // tipe data String
var umur = 20;      // tipe data int
```

Menggunakan tipe data eksplisit

Menyebutkan tipe data secara langsung

```
String namaLengkap = 'Ani Wijaya';
int jumlahProduk = 15;
double hargaSatuan = 99.99;
```

Aturan Penamaan Variabel

Aturan yang Harus Diikuti:

- Dimulai dengan huruf atau underscore (_)
- Dapat berisi huruf, angka, dan underscore
- Case sensitive (huruf besar dan kecil dibedakan)
- Tidak boleh mengandung spasi atau karakter khusus
- Tidak boleh menggunakan keyword Dart

Konvensi:

Menggunakan *camelCase* untuk penamaan variabel (direkomendasikan)

Keyword `final` dan `const`

final

Variabel yang nilainya hanya dapat diinisialisasi satu kali dan tidak dapat diubah setelahnya (saat runtime).

```
final String kota = 'Jakarta';
// kota = 'Bandung';
// Error!
```

const

Variabel yang nilainya sudah diketahui saat kompilasi (compile-time constant). Lebih ketat dari final.

```
const double PI = 3.14;
// const now = DateTime.now();
// Error!
```

Kapan Menggunakan `final` vs `const`?

final

- Gunakan saat nilai ditentukan saat runtime
- Contoh: hasil API, input pengguna, DateTime.now()
- Hanya dapat ditetapkan satu kali

```
final waktuLogin = DateTime.now();
```

const

- Gunakan saat nilai sudah diketahui saat kompilasi
- Lebih efisien untuk memori
- Tidak dapat menggunakan nilai yang ditentukan saat runtime

```
const int JUMLAH_HARI = 7;
const List BULAN = ['Jan', 'Feb'];
```

Tipe Data Primitif: Numbers

int

Bilangan bulat.

```
int usia = 25;
int tahun = 2023;
```

double

Bilangan desimal.

```
double tinggi = 175.5;
double phi = 3.14;
```

Konversi Tipe Number:

```
int angkaInt = 10;
double angkaDouble = angkaInt.toDouble(); // 10.0
double desimal = 10.5;
int bulat = desimal.toInt(); // 10
```

Tipe Data Primitif: Strings

String: Urutan karakter (teks)

```
// Dapat menggunakan tanda kutip tunggal atau ganda
String salam = 'Halo, Dunia!';
String pesan = "Belajar Dart itu menyenangkan.";

// Multi-line string menggunakan tiga tanda kutip
String puisi = '''Aku ingin mencintaimudengan sederhana...''';
```

String Interpolation

Menyisipkan variabel atau ekspresi ke dalam string menggunakan `$` dan `${}`.

```
String nama = 'Alice';
int usia = 30;
print('Nama: $nama, Usia: ${usia} tahun.');// Output: Nama: Alice, Usia: 30 tahun.

print('Panjang nama: ${nama.length} karakter.'); // Output: Panjang nama: 5 karakter.
```

Tipe Data Primitif: Booleans

`bool`: Merepresentasikan nilai kebenaran

Hanya memiliki dua kemungkinan nilai: `true` atau `false`

```
bool isAktif = true;
bool selesai = false;
```

Digunakan untuk:

- Kondisi logis dalam percabangan (if-else)
- Kondisi pada perulangan (while, do-while)
- Pengecekan status

Tipe Data Non-Primitif: List

List: Koleksi terurut dari objek (mirip array)

```
// List dengan tipe spesifik
List angka = [1, 2, 3, 4, 5]; // List of integers
List namaHari = ['Senin', 'Selasa', 'Rabu']; // List of strings

// List dinamis (campuran tipe data)
var campur = [1, 'dua', true]; // List dynamic
```

Operasi Dasar pada List

Mengakses elemen

```
List buah = ['apel', 'jeruk', 'mangga'];
print(buah[0]); // apel (indeks dimulai dari 0)
```

Menambah elemen

```
buah.add('anggur'); // [apel, jeruk, mangga, anggur]
buah.insert(1, 'pisang'); // [apel, pisang, jeruk,
mangga, anggur]
```

Menghapus elemen

```
buah.remove('jeruk'); // Menghapus nilai tertentu
buah.removeAt(0); // Menghapus berdasarkan indeks
```

Properti dan metode lain

```
print(buah.length); // Panjang list
print(buah.isEmpty); // Cek apakah kosong
print(buah.contains('mangga')); // Cek keberadaan elemen
```

Tipe Data Non-Primitif: Map

Map: Koleksi pasangan key-value (mirip dictionary/object)

```
// Map dengan tipe spesifik
Map kamus = { 'apple': 'apel', 'banana': 'pisang', 'orange': 'jeruk'};


// Map dengan nilai dinamis
Map user = { 'nama': 'Budi', 'umur': 25, 'isStudent': true};
```

Operasi Dasar pada Map

Mengakses nilai

```
Map kamus = {  'apple':
'apel',   'banana':
'pisang'};
print(kamus['apple']);
```

Menambah/mengubah pasangan

```
kamus['orange'] =
'jeruk';kamus['apple'
] = 'apel merah';
```

Menghapus pasangan

```
kamus.remove('banana'
);
```

Properti & metode

```
print(kamus.keys);
print(kamus.values);
print(kamus.length);
print(kamus.containsK
ey('apple'));
```

Operator Aritmatika

Digunakan untuk operasi matematika dasar

`+`	Penjumlahan
`-`	Pengurangan
`*`	Perkalian
`/`	Pembagian (hasil double)
`~/`	Pembagian integer
`%`	Modulo (sisa bagi)

Contoh Penggunaan:

```
int a = 10;
int b = 3;
print(a + b); // 13
print(a - b); // 7
print(a * b); // 30
print(a / b); // 3.3333...
print(a ~/ b); // 3
print(a % b); // 1
```

Operator Increment dan Decrement

Operator Increment (++)

Menambah nilai variabel sebesar 1

```
int a = 5;a++; // a sekarang 6
```

Pre-increment vs Post-increment:

```
int a = 5;
int b = a++; // b = 5, a = 6 (post-increment)
int x = 5;int y = ++x; // y = 6, x = 6 (pre-increment)
```

Operator Decrement (--)

Mengurangi nilai variabel sebesar 1

```
int a = 5;a--; // a sekarang 4
```

Pre-decrement vs Post-decrement:

```
int a = 5;
int b = a--; // b = 5, a = 4 (post-decrement)int x =
5;int y = --x; // y = 4, x = 4 (pre-decrement)
```

Operator Relasional (Perbandingan)

Membandingkan dua nilai dan menghasilkan nilai boolean

`==`	Sama dengan
`!=`	Tidak sama dengan
`>`	Lebih dari
`<`	Kurang dari
`>=`	Lebih dari atau sama dengan
`<=`	Kurang dari atau sama dengan

Contoh Penggunaan:

```
int x = 5;
int y = 10;
print(x == y); // false
print(x != y); // true
print(x > y);  // false
print(x < y);  // true
print(x >= 5); // true
print(y <= 9); // false
```

Hasil dari operator relasional selalu bertipe `bool` (true/false) dan sering digunakan dalam percabangan kondisi.

Operator Logika

&&

AND Logis: Mengembalikan `true` jika kedua operand bernilai `true`

```
bool hasPassed = true;
bool hasGoodGrade = true;
bool getScholarship = hasPassed
&& hasGoodGrade; // true
```

||

OR Logis: Mengembalikan `true` jika salah satu operand bernilai `true`

```
bool hasMoney = false;
bool hasScholarship = true;
bool canStudy = hasMoney ||
hasScholarship; // true
```

!

NOT Logis: Membalik nilai boolean (negasi)

```
bool isExpensive = true;
bool isAffordable =
!isExpensive; // false
```

Operator Penugasan (Assignment Operators)

Penugasan Dasar

```
int a = 10; // Menetapkan nilai ke variabel
```

Penugasan Kombinasi (Compound Assignment)

Menggabungkan operasi aritmatika dengan penugasan, seperti `+=`, `-=`, `*=`, `/=`, `~/=`, `%=`.

Contoh:

```
int x = 5;x += 3; // Sama dengan x = x + 3 (x jadi 8)
x *= 2; // Sama dengan x = x * 2 (x jadi 16)
x ~/= 4; // Sama dengan x = x ~/ 4 (x jadi 4)
```

Perbandingan Operasi:

`a += b`	`a = a + b`
`a -= b`	`a = a - b`
`a *= b`	`a = a * b`
`a /= b`	`a = a / b`
`a ~/= b`	`a = a ~/ b`
`a %= b`	`a = a % b`

Operator Null-aware

?? (Null-aware operator)

Memberikan nilai default jika variabel bernilai `null`.

```
String? nama;
String namaTampil = nama ??
'Tamu'; nama =
'Budi';namaTampil = nama ??
'Tamu';
```

?. (Null-safe access)

Mengakses properti atau method hanya jika objek tidak `null`.

```
String? pesan;
print(pesan?.length);
pesan = 'Halo';
print(pesan?.length);
```

??= (Null-aware assignment)

Menetapkan nilai ke variabel hanya jika variabel tersebut `null`.

```
String? pengguna;
pengguna ??= 'Anonim';
pengguna ??= 'User123';
```

Operator Type Test

Operator `is` dan `is!`

Memeriksa apakah objek adalah instance dari tipe tertentu

```
var data = 100;
print(data is int); // true
print(data is double); // false
print(data is! String); // true (bukan String)
```

Operator `as`

Melakukan type casting (konversi tipe)

```
dynamic nilai = 'Hello';
// Cast nilai sebagai String
String teks = nilai as
String;print(teks.toUpperCase()); // HELLO// Casting
yang salah akan melempar exception// int angka = nilai
as int; // Error!
```

Operator ini sangat berguna dalam pemrograman Dart, terutama saat berurusan dengan tipe dinamis dan polimorfisme.

Operator Kondisional

Operator Ternary (`? :`)

Alternatif singkat untuk pernyataan if-else.

```
// Sintaks: condition ? expr1 : expr2
int nilai = 75;
String hasil = nilai >= 70 ? 'Lulus' : 'Gagal';//
hasil = 'Lulus'
```

Contoh Lanjutan:

```
int point = 85;
String grade = point >= 90 ? 'A' :
point >= 80 ? 'B' :
point >= 70 ? 'C' : 'D'; // Simplified example//
grade = 'B'
```

Meringkas kode, namun bersarang berlebihan dapat mengurangi keterbacaan.

Operator Bitwise

Operator untuk manipulasi bit

& (AND)

```
print(5 & 3); // 1
```

| (OR)

```
print(5 | 3); // 7
```

^ (XOR)

```
print(5 ^ 3); // 6
```

~ (NOT)

```
print(~5); // -6
```

<< (Left Shift)

```
print(5 << 1); // 10
```

>> (Right Shift)

```
print(5 >> 1); // 2
```

Operator bitwise digunakan dalam pemrograman level rendah, seperti manipulasi flag dan optimasi performa.

Prioritas Operator

Urutan Prioritas (dari tertinggi ke terendah):

1. Unary (e.g., `++e`, `!e`)
2. Multiplicative (`*`, `/`, `%`)
3. Additive (`+`, `-`)
4. Shift (`<<`, `>>`)
5. Relational (`>`, `<`, `>=`, `<=`)
6. Equality (`==`, `!=`)
7. Logical (`&&`, `||`)
8. Conditional (`?:`)
9. Assignment (`=`, `*=`, dll.)

Contoh:

```
int hasil = 2 + 3 * 4; // 14// Perkalian (*) prioritas
lebih tinggi// Gunakan tanda kurung untuk mengubah
prioritas
int hasil2 = (2 + 3) * 4; // 20
```

Gunakan tanda kurung `()` untuk memperjelas dan menghindari kesalahan prioritas.

Studi Kasus 1: Konversi Suhu

Program mengkonversi suhu dari Celsius ke Fahrenheit dan Kelvin:

```
void main() {
double celsius = 30;
double fahrenheit, kelvin;
fahrenheit = (celsius * 9/5) + 32;
kelvin = celsius + 273.15;    print('$celsius°
C = $fahrenheit°F');  print('$celsius°C =
$kelvin K');}
```

Output:

```
30.0°C = 86.0°F30.0°C = 303.15 K
```

Studi Kasus 2: Kalkulator Sederhana

```
void main() {  // Deklarasi variabel
int angka1 = 10;
int angka2 = 5;  // Operasi aritmatika
int penjumlahan = angka1 + angka2;
int pengurangan = angka1 - angka2;
int perkalian = angka1 * angka2;  double
pembagian = angka1 / angka2;  int modulo =
angka1 % angka2;  // Menampilkan hasil
print('$angka1 + $angka2 = $penjumlahan');
print('$angka1 - $angka2 = $pengurangan');
print('$angka1 * $angka2 = $perkalian');
print('$angka1 / $angka2 = $pembagian');
print('$angka1 % $angka2 = $modulo');}
```

Output dan Penerapan Konsep:

Output program ini menunjukkan hasil operasi dasar aritmatika (penjumlahan, pengurangan, perkalian, pembagian, modulo) dari angka 10 dan 5. Contoh ini menggunakan variabel bertipe `int` dan `double`, berbagai operator aritmatika, serta `String interpolation` untuk menampilkan hasil.

```
10 + 5 = 1510 - 5 = 510 * 5 = 5010 / 5 = 2.010 % 5 = 0
```

Studi Kasus 3: Validasi Data Pengguna

```
void main() {
String username = 'budi123';
String password = 'pass123';
int umur = 17;
bool isUsernameValid = username.length >= 6;
bool isPasswordValid = password.length >= 6;
bool isAdult = umur >= 18;
bool canRegister = isUsernameValid && isPasswordValid;
bool canAccessAdultContent = canRegister && isAdult;
print('Dapat mendaftar: $canRegister');
print('Dapat mengakses konten:
$canAccessAdultContent');}
```

Output:

```
Dapat mendaftar: true
Dapat mengakses konten: false
```

Penerapan Konsep:

Variabel dengan tipe `String`, `int`, dan `bool`

- Operator relasional
- Operator logika
- String interpolation

Studi Kasus 4: Perhitungan Diskon

Program ini menghitung total diskon dan harga akhir. Diskon diberikan berdasarkan status member dan total belanja.

```
void main() {
double hargaBarang = 120000;
int jumlahBeli = 3;
String kodeMember = 'GOLD';
double total = hargaBarang * jumlahBeli;
double diskonPersen = 0;  if (kodeMember == 'GOLD') diskonPersen = 0.1;  else if (kodeMember == 'SILVER') diskonPersen =
0.05;
double diskonTambahan = total > 300000 ? 0.05 : 0;
double totalDiskon = total * (diskonPersen + diskonTambahan);
double hargaAkhir = total - totalDiskon;
print('Total: Rp$total');
print('Diskon: Rp$totalDiskon');
print('Akhir: Rp$hargaAkhir');}
```

Output:

```
Total: Rp360000
Diskon: Rp54000.0
Akhir: Rp306000.0
```

Penerapan Konsep:

- Variabel dan operasi matematika
- Operator kondisional (ternary)
- Logika pilihan (if-else)
- String interpolation

Studi Kasus 5: Status Kelulusan

```
void main() {
Map nilaiMahasiswa = {    'Matematika': 85,    'Fisika': 75,    'Pemrograman': 90,  };
double total = 0;  nilaiMahasiswa.forEach((matkul, nilai) {    total += nilai;  });
double rataRata = total / nilaiMahasiswa.length;
String status = rataRata >= 60 ? 'LULUS' : 'TIDAK LULUS';
String predikat;  if (rataRata >= 90) {    predikat = 'A';  } else if (rataRata >= 80) {
predikat = 'B';  } else if (rataRata >= 70) {    predikat = 'C';  } else {    predikat =
'D/E';  }}
```

Output:

```
Nilai Mahasiswa:
Matematika: 85
Fisika: 75
Pemrograman: 90
Bahasa Inggris: 80
Rata-rata: 82.5
Status: LULUS
Predikat: B
```

Studi kasus ini menunjukkan penggunaan `Map` untuk nilai, perhitungan rata-rata, operator kondisional (ternary), dan struktur `if-else if` untuk penentuan status dan predikat kelulusan.

Perbandingan dengan Java

Kemiripan

- Sintaks dasar (kurung kurawal, titik koma)
- Operator aritmatika, relasional, dan logika
- Konsep variabel dan tipe data primitif
- Berorientasi objek

Perbedaan

Type inference dengan `var` di Dart

Dart tidak memiliki keyword `new` yang wajib

- Fitur null safety lebih canggih di Dart
- String interpolation lebih mudah di Dart
- Collection literals lebih simpel di Dart

Jika Anda sudah mengenal Java, belajar Dart akan terasa lebih mudah karena banyak konsep dasar yang mirip meskipun Dart memiliki fitur modern yang membuat penulisan kode lebih efisien.

Perbandingan dengan JavaScript

Kemiripan

Type inference dengan `var`

- Fungsi dapat disimpan dalam variabel
- Operator kondisional dan logika mirip
- String interpolation (meski sintaks berbeda)

Perbedaan

- Dart mendukung deklarasi tipe statis
- Lebih terstruktur, lebih mirip Java/C#
- Dart memiliki null safety lebih baik
- Sintaks class lebih konsisten di Dart
- Dart tidak memiliki prototype inheritance

Dart dirancang untuk mengatasi kelemahan JavaScript dengan menambahkan tipe statis opsional dan struktur yang lebih kuat, sambil mempertahankan produktivitas JavaScript.

Praktik Terbaik: Penamaan Variabel

✅ Gunakan:

```
// camelCase untuk variabel dan fungsi
String namaLengkap = 'Budi Santoso';
double hitungLuasLingkaran(double radius) {...}
// PascalCase untuk nama kelas
class MahasiswaBaru {...}
// SCREAMING_SNAKE_CASE untuk konstanta
const double PI = 3.14159;
const int HARI_DALAM_MINGGU = 7;
```

❌ Hindari:

```
// Nama tidak deskriptif
var a = 10;
var xyz = 'Budi';
// Inkonsistensivar Nama
User = 'Ani';
var user_age = 25;
// Terlalu panjang
var nilaiRataRataUjianAkhirSemesterGanjilTahunAjaran20222023
= 85.5;
```

Variabel yang diberi nama dengan baik akan membuat kode lebih mudah dibaca, dipahami, dan dipelihara. Nama yang baik menjelaskan "apa" yang disimpan, bukan "bagaimana" itu digunakan.

Praktik Terbaik: Penggunaan `final` dan

Gunakan `final` secara default

Selalu gunakan `final` untuk variabel yang nilainya tidak akan berubah setelah inisialisasi. Ini membantu mencegah bug akibat perubahan nilai yang tidak disengaja.

```
final String nama = 'Budi';
final double phi = 3.14;
```

Gunakan `const` untuk konstanta compile-time

Untuk nilai yang dapat ditentukan saat kompilasi, gunakan `const` untuk meningkatkan performa.

```
const int HARI_DALAM_MINGGU = 7;
const List BULAN = ['Jan', 'Feb',
'Mar'];
```

Hindari variabel global yang mutable

Jika membutuhkan variabel global, usahakan untuk membuatnya `final` atau `const` untuk menghindari perubahan keadaan yang tidak terduga.

Praktik Terbaik: Penggunaan Tipe Data

Pilih tipe data yang sesuai

Gunakan tipe data yang paling tepat untuk kebutuhan aplikasi:

`int` untuk angka bulat

`double` untuk angka desimal

`String` untuk teks

`bool` untuk nilai kebenaran

Manfaatkan collection dengan tipe spesifik

Tentukan tipe untuk collection agar lebih aman:

```
List nama = ['Ani', 'Budi',
'Citra'];
Map umur = {'Ani': 25, 'Budi':
30};
```

Hindari tipe dynamic jika memungkinkan

Menggunakan tipe dynamic menghilangkan manfaat static type checking yang menjadi kelebihan Dart.

```
// Hindari:
dynamic data = 'Hello';
// Lebih baik:
String data = 'Hello';
```

Praktik Terbaik: Null Safety

Apa itu Null Safety?

Fitur Dart yang membantu mencegah error akibat nilai null dengan membedakan tipe yang bisa null dan tidak bisa null.

Cara Menggunakan:

```
// Variabel non-nullable (tidak bisa null)
String nama = 'Budi';
int umur = 25;
// Variabel nullable (bisa berisi null)
String? alamat; // Nilai default adalah null
int? tinggi = 170; // Bisa diubah menjadi null nanti
```

Operator Null Safety:

```
// Null-aware operator
(??)String alamat = alamatUser ?? 'Alamat tidak
diketahui';
// Null-aware access (?.)
int? panjangNama = nama?.length;
// Null assertion operator (!)
// Gunakan hanya jika yakin nilai tidak null
int panjang = nama!.length;
// Null-aware cascade operator (?..)
user?..nama = 'Budi' ..umur = 25;
```

Praktik Terbaik: Menghindari Masalah

Waspadai Integer Division

Perhatikan pembagian integer vs. double

```
// Mungkin tidak sesuai harapan
int hasil = 5 / 2; // hasil = 2 (bukan 2.5)// Solusi
double hasil = 5 / 2; // hasil = 2.5
```

Hindari perbandingan floating-point langsung

Perbandingan langsung bisa tidak akurat karena representasi floating-point

```
// Hindari
if (0.1 + 0.2 == 0.3) {...}
// mungkin false!
// Lebih baik
if ((0.1 + 0.2 - 0.3).abs() < 0.000001) {...}
```

Perhatikan operasi pada variabel nullable

Gunakan operator yang tepat untuk menangani nilai null

```
String? nama;//
print(nama.length);
// Error!print(nama?.length);
// Aman, hasilnya null
```

IDE dan Tools untuk Dart

Visual Studio Code

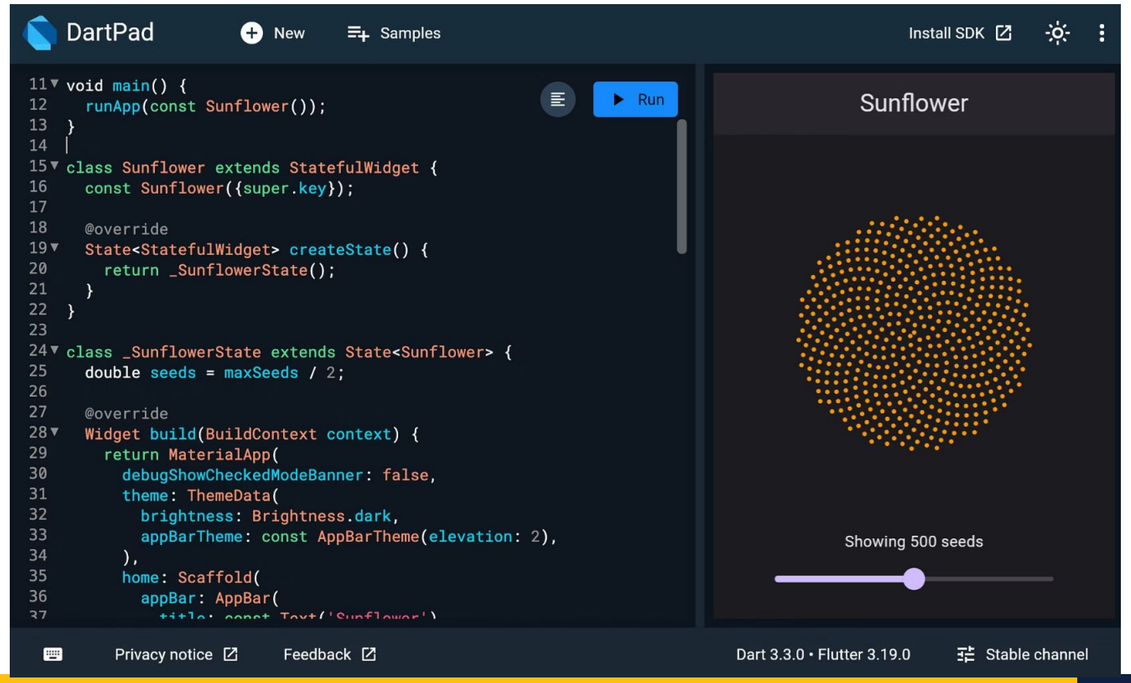
- Gratis dan ringan
- Extension Dart dan Flutter tersedia
- Integrasi dengan Dart Analyzer
- Fitur hot reload

Android Studio / IntelliJ IDEA

- IDE lengkap dengan plugin Dart dan Flutter
- Tools debugging yang kuat
- Integrasi emulator
- Fitur refactoring kode

DartPad

- IDE online untuk Dart
- Tidak perlu instalasi
- Cocok untuk belajar dan eksperimen cepat
- https://dartpad.dev




```
11 void main() {
12   runApp(const Sunflower());
13 }
14
15 class Sunflower extends StatefulWidget {
16   const Sunflower({super.key});
17
18   @override
19   State<StatefulWidget> createState() {
20     return _SunflowerState();
21   }
22 }
23
24 class _SunflowerState extends State<Sunflower> {
25   double seeds = maxSeeds / 2;
26
27   @override
28   Widget build(BuildContext context) {
29     return MaterialApp(
30       debugShowCheckedModeBanner: false,
31       theme: ThemeData(
32         brightness: Brightness.dark,
33         appBarTheme: const AppBarTheme(elevation: 2),
34       ),
35       home: Scaffold(
36         appBar: AppBar(
37
```

Sumber Belajar Lanjutan

Dokumentasi Resmi

dart.dev - Situs resmi dengan tutorial, contoh kode, dan dokumentasi API lengkap tentang Dart

Buku

"Dart Apprentice" oleh Jonathan Sande, "Programming in Dart: Basics and Fundamentals" oleh Balaji Venkatesh

Video Tutorial

Kanal YouTube "Flutter" resmi, kursus Udemy "Complete Dart Guide" oleh Maximilian Schwarzmüller

Contoh Kode

GitHub repository "awesome-dart" dengan kumpulan contoh kode dan sumber daya belajar Dart

Gunakan sumber-sumber ini untuk memperdalam pemahaman tentang Dart dan meningkatkan keterampilan pemrograman Anda.

Tugas Praktikum

Tugas Individual:

1. Buat program Dart sederhana untuk menghitung BMI (Body Mass Index) berdasarkan berat dan tinggi yang diinput
2. Buat kalkulator konversi mata uang dengan minimal 3 jenis mata uang
3. Buat program yang menerapkan semua jenis operator yang telah dipelajari dalam satu aplikasi konsol

Kriteria Penilaian:

- Ketepatan implementasi variabel, tipe data, dan operator
- Kemampuan menerapkan praktik terbaik
- Kejelasan dan kerapian kode
- Dokumentasi kode (komentar)

Tantangan Tambahan:

Buat aplikasi konversi unit (panjang, massa, volume, suhu) dengan fitur:

- Menu pemilihan kategori konversi
- Minimal 5 unit untuk setiap kategori
- Validasi input (hindari nilai negatif untuk massa, volume, dll.)
- Tampilkan hasil dengan format yang rapi
- Implementasikan penggunaan Map untuk menyimpan faktor konversi

Rangkuman

Variabel di Dart

Deklarasi dengan `var` atau tipe eksplisit, `final` untuk nilai tetap saat runtime, `const` untuk nilai tetap saat kompilasi.

Tipe Data

Primitif: `int`, `double`, `String`, `bool`. Non-primitif: `List`, `Map`, dan lainnya.

Operator

Aritmatika, relasional, logika, penugasan, null-aware, dan lainnya untuk manipulasi data dan kondisi.

Praktik Terbaik

Penamaan yang baik, penggunaan `final`/`const`, menerapkan null safety, dan penggunaan tipe data yang tepat.

Pemahaman variabel, tipe data, dan operator adalah fondasi untuk pengembangan logika pemrograman yang lebih kompleks dalam aplikasi mobile.